

Lab Class : 1

Brief History of Oracle

What / Why / When / How / Where about Oracle ?

Oracle Corporation was founded in 1971 in Redwood, California.

- They introduced the First Relational Database Management System based on IBM system.
- Oracle is the First database Management System utilizing IBM'S structured Query Language (SQL) technology.
- Oracle has 42,000 professionals in 93 countries around the world today.

Where did the word Oracle Originate from ?

Larry Ellison and Bob Miner were working on a consulting project for CIA (Central Intelligence Agency in USA) Where CIA wanted to use SQL Language. The name of the project was ORACLE.

What is the origin of the name ORACLE ?

Er. is an abbreviation for Engineer and SUN is an acronym for Stanford University Network. However, the word ORACLE is neither an abbreviation nor an acronym (see an English dictionary for the meaning of the word **ORACLE**). It appears in uppercase because this is the branding style Oracle corporation chose for itself.

The word oracle means:

- Prophecy or prediction
- the answer to a question believed to come from the Gods
- A statement believed to be infallible and authoritative
- A shrine at which these answers are given

So why is it named Oracle?

Larry Ellison and Bob Miner were working on a consulting project for the CIA (Central Intelligence Agency in USA) where the CIA wanted to use this new SQL Language. The code name for the project was Oracle (the CIA saw this as the system to give all answers to all questions for something such.)

The project eventually died but Larry and Bob saw the opportunity to take what they had started and market it. So they used that project's code name of Oracle to name their new RDBMS engine. Funny thing is that the CIA was one of Oracle's first customers.

ORACLE:- In Greece, a place where people could go to ask the Gods for advice or information about the future.

Oracle Data Types

- I. **number** :Integer datatype with default length of digits.
- II. **number(n)** :Integer datatype with specified length of digits
- III. **char(n)** :- Fixed length character strings e.g. 'Anu' .Fixed length character data, Maximum size **2000** bytes.

In this type of variable blank is padded to the specified length.
- IV. **varchar(n)** :- Variable length character string having a maximum size of **4000** bytes.
- V. **varchar2(n)** :- Variable length character strings, automatically trims the leading and trailing blanks . e.g. 'Raj'
- VI. **number(n,d)** :- Fixed and floating point no.

n - Precision

d - Scale for decimal .e.g. 99.26
- VII. **date** - Date and time
- VIII. **long** - Variable length character type up to **2 GB** of information.

e.g. 'abcd.....' up to 2GB
- IX. **Nchar** :- Multibyte
- X. **Blob** :- Binary large object upto **4 GB** in length
- XI. **Clob** :- Character large object upto **4 GB** in length

Lab Class : 2

SQL STATEMENTS

1. CREATING A TABLE

SQL > create table student (roll number, name varchar2(30), mark number);

2. INSERTING RECORD IN A TABLE

SQL > insert into student values (101,'Sachin',87);

3. TO VIEW THE CONTENT OF ONE COLUMN OF A TABLE

SQL> select roll from student;

4. TO VIEW THE CONTENTS OF THE TABLE

SQL > select * from student;

5. TO VIEW THE CONTENTS IN SPECIFIC ATTRIBUTES OF THE TABLE

SQL > select roll, mark from student;

6. TO VIEW SPECIFIC ROWS FROM A TABLE

SQL > select * from student where mark>80;

SQL > select roll, mark from student where mark>80;

SQL > select roll, mark from student where mark>60 and mark<70;

N.B. – “where” clause is used to impose a condition on the query

7. TO VIEW THE SCHEMA OF THE TABLE

SQL > desc student;

8. TO VIEW ALL THE TABLES PRESENT IN THE DATABASE

SQL > select * from tab;



8.a. INSERTING DATA INTO A TABLE FOR LESS NUMBER OF COLUMNS THAN ACTUALLY PRESENT IN THE TABLE

SQL> insert into student(roll,name) values(105,'Dipak');

Lab Class : 3

9. TO COUNT THE NUMBER OF RECORDS IN A TABLE

SQL > select count(*) from student;

N.B. Count() is a function to count the number of rows.

10. TO VIEW THE RECORDS IN A QUERY AVOIDING THE DUPLICATE RECORDS

SQL> select distinct * from student;

SQL> select distinct name from student;

SQL> select count (distinct stream), count(stream), count(*) from student;

11. “ORDER BY” CLAUSE

“order by” clause is used to sort the result of a query in ascending or descending order of column value, column number etc.

SQL> select roll, mark from student order by mark;

SQL> select roll, mark from student order by mark desc;

12. “GROUP BY” CLAUSE

“group by” clause in a select statement divides the rows of a queried table or view into smaller groups and displays the output in a particular sequence.

SQL> select stream, count(*) No_of_Students from student group by stream order by No_of_Students;

<u>stream</u>	<u>No of Students</u>
MCA	31
MBA	17
COSTING	10
PART(MBA)	12

13. “HAVING CLAUSE”- It operates in same fashion as that of where clause, except that the “having” clause operates on group functions(“GROUP FUNCTION” is a function which operates on multiple rows) where as “where” clause operates on individual rows.

SQL> select stream, count(*) No_of_Students from student group by stream having count(*) > 15;

<u>stream</u>	<u>No of Students</u>
MCA	31
MBA	17

14. “UPDATE” Statement :- It is used to modify or update an already existing row of a table.

SQL> update student set name = ‘Sachin Ramesh’ where roll=101;

15. “DELETE” Statement :- It is used to delete already existing row or rows from a table .

SQL> delete from student where roll=101;

16. ALIAS FOR THE COLUMN NAME

SQL> select roll ROLL_NO, name NAME_OF_THE_STUDENT from imit;

<u>ROLL_NO</u>	<u>NAME_OF THE STUDENT</u>
101	Sachin

17. TO ADD A NEW COLUMN TO AN EXISTING TABLE

SQL> alter table imit add(stream varchar2(20));

18. TO MODIFY THE DATATYPE OF AN EXISTING COLUMN (ONLY UPWARD TYPE)

SQL> alter table imit modify(stream varchar2(30));

N. B. : “upward type” means if the datatype of column is number it can be modified to varchar2 type. Similarly, a smaller sized varchar2() can be modified to larger size. But, the reverse isn’t allowed.

19. TO DROP A COLUMN FROM A TABLE

SQL> alter table imit drop column stream;

20. TABLE ALIAS

SQL> select a.roll, a.name from imit a where a.mark>60;

Lab Class : 4

PSEUDO COLUMNS IN A TABLE

In addition to the existing columns created by the user in a table some extra columns are present in every table. These columns are known as Pseudo Columns.

Such as :

- a) rownum
- b) rowid
- c) sysdate etc...

21. TO DISPLAY THE ROW NUMBER IN A TABLE

SQL > select rownum, roll, name from imit;

22. TO DISPLAY THE SYSTEM DATE IN A TABLE

SQL> select roll, name, sysdate from imit;

23. TO DISPLAY THE ROW ID IN A TABLE

SQL> select rowid, roll, name from imit;

SQL OPERATORS

- i. Arithmetic operators
- ii. Character Operators
- iii. Comparison Operators
- iv. Logical Operators
- v. Set Operators

I. Arithmetic Operators :

(a) Unary:-

24. SQL> Select country ,temp from weather where temp <-6

(b) **Binary:-** (*,/,+,-)

25.SQL> Select roll, mark+5 from imit;

II. Character Operators:

Character operators are used to manipulate strings. The most common character operator is the concatenation (||) operator.

26.SQL> Select name || ' has marks ' || mark from imit;

III. Comparison operators :

- a) =
- b) !=,<>, ^= Inequality
- c) <
- d) >=
- e) <=
- f) in : Equal to any member in a set

27. SQL> select roll, name, mark from imit where city in ('CTC','BBSR');

- g) all : compares a value to every value in a list between x and y
- h) like : Returns true when the first expression matches the pattern of 2nd expression.

28. SQL> select roll, name, mark from imit where name like 's%';

IV. Logical Operator :

A logical operator is used to produce a single result from combining the 2 separate conditions.

- a) and
- b) or
- c) not

29. SQL>select roll, name, mark from imit name like 'a%' or mark > 70;

V. Set Operator :

Set operator combines the result of 2 separate queries into a single result.

- a) UNION- Returns all distinct rows from both queries .

30. SQL> select roll, name from imit where city='CTC' union select roll, name from imit where stream='MCA';

- b) UNION ALL- Returns rows from both queries (including duplicate rows)

31. SQL> select roll, name from imit where city='CTC' union all select roll, name from imit where stream='MCA';

- c) INTERSECT- Returns all rows selected by both queries (common rows)

32. SQL> select roll, name from imit where city='CTC' intersect select roll, name from imit where stream='MCA';

- d) MINUS- Returns all distinct rows that are in the first query but not in the second one.

33. SQL> select roll, name from imit where city='CTC' minus select roll, name from imit where stream='MCA';

Operator Precedence

Precedence defines the order that Oracle uses when evaluating the different operate in the same expression.

1. - SQL * plus continuation command
2. & Parameter prefix
3. && Substituion prefix
4. : Prefix for host variable
5. . Variable separator
6. () Surrounds sub queries
7. ' ' Single quote
8. " " Double quote
9. @ Replace
10. () Overrides the normal operator
11. +- Arithmetic
12. *, / Multiplication and Division
13. +, - Addition and Subtraction
14. || Concatenation



INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY, CUTTACK
DEPARTMENT OF MCA
MCA 1ST SEMESTER DATABASE ENGINEERING LAB

- 15. not
- 16. and
- 17. or
- 18. union
- 19. intersect
- 20. minus

Lab Class : 5

SQL functions

Group Functions :

A group function returns a single result row for a group at queried rows. Group value functions ignore NULL values and calculate the result without them.

- i. count (value) :- count of rows for a column.
- ii. sum (value) :- sum of all values for group of rows
- iii. avg (value) :- average of all values for group of rows
- iv. max (value) :- maximum of all values for group of rows
- v. min (value) :- minimum of all values for group of rows

34. SQL>select count(mark) TOTAL , sum(mark) SUM, avg(mark) AVERAGE, max (mark) MAXIMUM , min(mark) MINIMUM from imit;

Single row functions:

A single row function returns a single result row for every row of a queried table or view.

Single row function can be categorized as

- i. **Character functions.**
- ii. **Number functions**
- iii. **Date functions**
- iv. **Conversion functions**

I. CHARACTER FUNCTIONS :

Character functions take character arguments.

a) **initcap** :- This function returns the string, the first letter of each word in upper case and all other letters in lower case

35. SQL> select initcap(name) from imit;

b) **instr** :- This function allows searching for a set of characters. It tells the position of the searched set of characters in the given string.

36. SQL> select instr (name, 'a') from imit;

c) **substr** :- It is used to extract data from a longer text string.

37. SQL> select name, substr(name,2) OUTPUT from imit;

<u>Name</u>	<u>OUTPUT</u>
Manisha	anisha
Pooja	ooja
Ranjeet	anjeet

d) **upper** :- It takes any string or column and converts all letters in it into uppercase

38. SQL > select upper(name) from imit where mark>67;

upper(name)
ANJAN
JYOTI
GYAN

e) **rpadd** :- It allows us to pad the right side of a column with any set of character.

39. SQL > select rpadd (name,10, '*') NAME from imit where mark>80;

NAME

Sachin*****

f) length :- This function returns a numeric value which is the length of the argument.

40. SQL> select name, length (name) LENGTH from imit where mark>60;

<u>Name</u>	<u>LENGTH</u>
Sachin	6

II.Number function

Number function accepts numeric input and returns numeric values.

a) abs :- (Absolute value):-

Absolute value is the measure of the magnitude of something. It always gives a positive no.

41. SQL> select abs(-234) from dual;

<u>ABS(-</u>	
<u>234)</u>	234

b) ceil :- it produces the next integer if the argument isn't integer else it produces the same value.

42. SQL > select ceil(8.10),ceil(-9),ceil(-4.01),ceil(3) from dual;

<u>ceil(8.10)</u>	<u>ceil(-9)</u>	<u>ceil(-</u>	<u>ceil(3)</u>
<u>4.01)</u>	<u>ceil(3)</u>		
9	-9	-4	3

c) floor :- it produces the previous integer if the argument isn't integer else it produces the same value.

43. SQL> select floor (6), floor (-6), floor(2.22), floor(-2.22) from dual;

<u>floor (6)</u>	<u>floor (-6)</u>	<u>floor(2.22)</u>	<u>floor(-2.22)</u>
6	-6	2	-3

d) mod :- Modulus is a fun which divides a value by a divisor and gives us the remainder .

44. SQL>select mod(100,00) , mod (2.3,0.2) , mod(33,38) from dual;

mod(100,00) mod (2.3,0.2) mod(33,38)

0 .1 33

e) **sqrt:-** Finds the square root of a given value

45. SQL>select sqrt (100), sqrt(3.6) from dual;

sqrt (100) sqrt(3.6)

10 1.897366

f) **sign:-**It gives the sign of the value but not it's magnitude.

46. SQL > select sign (600) , sign(-600), sign(0) from dual;

sign(600) sign(-600) sign(0)

1 -1 0

g) **round & trunc :-** trunc chops off digits of precision from a no.

round rounds numbers to a given no. of digits of precision

.

47. SQL>Select trunc(33.666666666,2), round(33.6666666,2) form dual;

trunc(33.666666666,2) round(33.6666666,2)

33.66 33.67

h) **ASCII Function:-** Returns the ASCII equivalent code of a character.

48. SQL >select ascii ('D') from dual;

ascii ('D')

68

(3) Date function

In order to work with date datatypes , let's add a new column to imit, i.e. dob;

49. SQL> alter table imit add(dob date);

Now let's enter some data for the dob attribute.

50. SQL> insert into imit values(101,'sachin','16-sep-1985');

Sysdate is a column that always returns the current date & the time and can be used anywhere . It can be considered as a (hidden) pseudo column present in every table .

I. **add_months**:- The function has 2 arguments ,a date and an integer representing the no. of months . It returns a date equal to the given date plus the specified no. of months.

51. SQL> select roll, name, add_months(dob,48) from imit;

SQL>select doj, add_months(dob,6) from imit;

II. **months_between (date1,date 2)** : It returns the months between date1 and date2.

52. SQL> select roll, name, months_between(dob, sysdate) from imit;

III. **last_day(date)** : It gives date of last day of month that day is in.

53. SQL> select roll, name, last_day(dob) from imit;

IV. **greatest(date1,date2,date3,...)** : Picks latest date from list of dates.

54. SQL> select least('15-sep-2008','16-aug-2009', '21-dec-2007') from dual;

V. **least(date1,date2,date3,...)** : Picks earliest date from list of dates.

55. SQL> select least('15-sep-2008','16-aug-2009', '21-dec-2007') from dual;

VI. **next_day(date,'day')** : Gives date of next day after date, where 'day' is 'Monday', 'Tuesday', and so on.

56. SQL> select next_day(dob, 'Friday') from imit;

Lab Class : 6

(4) Conversion Function :-

Conversion function converts a value from one datatype to another.

- a) to_char
- b) to_number
- c) to_date

a) to_char

to_char (n[,format])

It converts a value of a number data type to a character data type , using the optional format string .

57. SQL>Select to_char (17145, '\$ 099,999') "char" from dual ;

char

\$ 017 ,145

to_char (date[,format])

It converts a value of a date data type to char value .

58. SQL>select 'Rs'||damout||'deposited on'||to_char (ddate ,'ddspth,month, yyyy')||'in account no.:'||acc_no DEPOSIT_DETAILS from bank;

DEPOSIT DETAILS

o/p: Rs. 10020 deposited on Eleventh ,November, 2009 in account no.: 1001

Parameter	Explanation
YEAR	Year, spelled out
YYYY	4-digit year
YYY YY	Last 3, 2, or 1 digit(s) of year.

Y	
IYY IY I	Last 3, 2, or 1 digit(s) of ISO year.
IYYY	4-digit year based on the ISO standard
RRRR	Accepts a 2-digit year and returns a 4-digit year. A value between 0-49 will return a 20xx year. A value between 50-99 will return a 19xx year.
Q	Quarter of year (1, 2, 3, 4; JAN-MAR = 1).
MM	Month (01-12; JAN = 01).
MON	Abbreviated name of month.
MONTH	Name of month, padded with blanks to length of 9 characters.
RM	Roman numeral month (I-XII; JAN = I).
WW	Week of year (1-53) where week 1 starts on the first day of the year and continues to the seventh day of the year.
W	Week of month (1-5) where week 1 starts on the first day of the month and ends on the seventh.
IW	Week of year (1-52 or 1-53) based on the ISO standard.
D	Day of week (1-7).
DAY	Name of day.
DD	Day of month (1-31).
DDD	Day of year (1-366).
DY	Abbreviated name of day.
J	Julian day; the number of days since January 1, 4712 BC.
HH	Hour of day (1-12).
HH12	Hour of day (1-12).
HH24	Hour of day (0-23).
MI	Minute (0-59).
SS	Second (0-59).
SSSSS	Seconds past midnight (0-86399).
FF	Fractional seconds. Use a value from 1 to 9 after FF to indicate the number of digits in the fractional seconds. For example, 'FF4'.
AM, A.M., PM, or P.M.	Meridian indicator
AD or A.D	AD indicator
BC or B.C.	BC indicator
TZD	Daylight savings information. For example, 'PST'
TZH	Time zone hour.

TZM	Time zone minute.
TZR	Time zone region.

Applies To:

- Oracle 8i, Oracle 9i, Oracle 10g, Oracle 11g

For example:

`to_date('2003/07/09', 'yyyymmdd')` would return a date value of July 9, 2003.
`to_date('070903', 'MMDDYY')` would return a date value of July 9, 2003.
`to_date('20020315', 'yyyymmdd')` would return a date value of Mar 15, 2002.

b) to_number(char)

This function converts char , a character value expressing a number to a NUMBER data type

59. SQL> select to_number (substr ('Rs7500', 3,4)) from dual ;

60. SQL >Select to_number ('1234') from dual;

c) to_date(char[,format])

It converts a character field to a date field.

61. SQL> select to_char(to_date('25jun20051134p.m.', 'ddmonyyyyhhmia.m.'), 'ddd-q-mi-am-hh24-yyyy,y,ddspth,mmspth,yyyyspth') from dual;

SQL> select to_char(to_date('25-jun-1998', 'dd-mon-yyyy'), 'ddd,month,y----q:::mm') from dual

SQL> select to_char(sysdate, 'ddd-month-yy') from dual;

The date data type is used to store date and time information. It stores the information about the followings.

- Century
- Year
- Month
- Day

- e. Hour
- f. Minute
- g. Second

WRITE THE SQL QUERIES FOR THE FOLLOWING QUESTIONS FROM IMITEMP TABLE.

IMITEMP

Empno	ename	city	des	Dob	doj	Dept	Sal
1	Sachin	bbsr	Prof	7.02.1979	4.06.2001	Comp	48000
2	Saurav	ctc	Lect	8.06.1982	4.04.2002	Elect	32000
3	Saurav	sbp	Lect	2.06.1972	8.06.2001	Elect	18000
4	Sunil	ctc	Prof	8.02.1971	7.09.2001	Comp	25000
5	Ajit	rkl	Clerk	12.11.1972	8.02.2003	Civil	20000

- 1) Display the employee name and salary whose salary > 19000.
- 2) Display the empname, salary and increased by 15%. Label the new column as new salary.
- 3) Modify the query to add a column that will subtract old salary from new salary .Label the column as increase .
- 4) For each employee display the ename & calculate no. of months between today and the doj label the column as months_worked. Display the result in the ascending order of months worked . Round up the months.
- 5) Write a query that will display the employee name with the first letter capitalized and of their name for all employees whose name starts with J, A or S.
- 6) Display name & salary for employees. Format salary to 15 characters long left padded with *.
- 7) Display the name & age of employees where age > 28 years .Format name to 15 characters long right padded with *.
- 8) Display the highest, lowest, sum, average salary of all employees. (Round the results to nearest whole numbers.)
- 9) Modify the query to display the min, mix, sum & average salary for each job type.



- 10) Display the number of people with same job.
- 11) Display the difference between highest is lowest salaries label the column as difference .
- 12) Display the department name, number_of_employees in that department and average salary of that department.
- 13) Display ename & salary of the employees whose salary > average salary. Sort the result in the order of salary.
- 14) Display the name of the employees in computer science whose starts with J.
- 15) Display the name of those employees whose name starts with r and ends with h.
- 16) Display the empno and name of those whose salary is greater than the salaries of computer sc dept.
- 17) Display the empno and name of those whose salary is greater than the average salary of their own department.
- 18) Display the number of people in the same department.
- 19) Display the name of all the employees whose age is greater than 30 years.
- 20) Display the name of all the employees whose 3rd character of the name is c.
- 21) Display the name and age (in years) of all employees.
- 22) Display the name of all the employees who are from computer science department or city is either Cuttack, Bhubaneswar or Sambalpur.

Lab Class : 7

Query in Multiple Tables

SQL allows us to draw information from multiple tables using JOINS.

Definition : A join is a query in which data is retrieved from more than one table .

- Joins are the foundation of the multi table query processing in SQL.
Different types of joins that can be made between tables are :-

1. Cartesian Join
2. Equi Join
3. Non equi Join
4. Self Join
5. Outer Join
6. Natural Join
7. Theta Join
8. Semi Join

Cartesian Join

When we don't specify any condition through WHERE clause, each row of the table is matched with every row of other table. This results in Cartesian join. If table 1 has n rows and table2 has m rows, the total no. of rows produced in result will be = n*m rows.

BORROWER	
Cust_name	Loan_no
Sachin	L-15
Saurav	L-16

Loan		
Loan_no	Br_name	Amount
L-15	BBSR	25k
L-16	CTC	35k
L-17	BBSR	40k

62. SQL>select borrower.Cust_name, borrower.loan_no, loan.loan_no, loan.br_name, loan amount from borrower,loan;

Cust_name	loan_no	loan_no	br_name	amount
Sachin	L-15	L-15	BBSR	25k

Equi Join

Equi joins are formed as a result of an exact match between 2 columns. In this, the joining condition is based on the equalities between the values of the common columns.

**63. SQL>select brrower.cust_name, loan.br_name from borrower, loan where
borrower.loan_no=loan.loan_no;**

<u>cust-name</u>	<u>br-name</u>
Sachin	BBSR
Saurav	CTC

Non-Equi Join

The non-equi join is a join where the link condition isn't a direct relationship, but is inferred in some fashion it is any join where the "=" symbol isn't the joining condition.

emp	
Emp-no	Sal
101	800
102	1100
103	1250
104	1600
105	1500
106	2975
107	2850
108	8560

salgrade		
grade	hisal	Losal
1	1200	700
2	1400	1201
3	2000	1401
4	3000	2001
5	9000	3001

64. SQL> select emp_no, sal, grade, hisal, losal, from emp, salgrade where sal between losal and hisal;

N.B. These above 2 tables have no common column.

Self Join

A join that relates to itself. This is helpful if the join is to be between one row in a table and another row in the same table.

To list all employee numbers, salary having salary greater than or equal to the salary of emp_no 105.

65. SQL> select a.emp, b.sal from depositor and depositor b where a . sal >=b.sal and b.emp_no=105;

Outer Join

Outer join is used to include mismatched rows from one table. The symbol (+) is added at the end of the column name to create the outer join.

Account		
Acc-no	Name	type
101	Sachin	CA
102	Saurav	SA
103	Rahul	CA
104	Ajit	SA

Deposit	
Acc-no	Balance
101	25k
102	35k
103	40k

66. SQL>select a. acc_no , name , b.balance from account a , deposit b where b. acc-no (+) = a.acc_no;

acc-no	Name	Balance
101	Sachin	25k

102	Saurav	35k
103	Rahul	40k
104	Ajit	

The left outer join takes all tuples in the left relation that don't match with any tuple in the right relation, pads the tuples with NULL values for all other attributes from the right relation, and adds them to the result of the natural join .

- The right outer join pads from the right relation that don't match any from the left relation with NULL and adds them to the result of the natural join.
- The full outer join does both of these operations , padding tuples from the left relation that don't match any from the right relation ,as well as the tuples from the right relation that don't match any from the left relation adding them to the result of the join.

Lab Class : 8

SUBQUERIES

A sub query makes it possible for a user to base the search criteria of one select statement on the results of another SELECT statement.

Sub queries must always be enclosed in parenthesis.

Oracle first evaluates the nested query then the main query is executed.

67. To find the second maximum salary from table emp.

SQL> Select max(sal) from imitemp where sal <(select max(sal) from imitemp);

68. To find the empnos from imitemp table whose sal > avg.salary .

SQL> Select empno from imitemp where sal > (select avg (sal) from imitemp);

69. To display acc-no, name of account holder and account type whose account type is same as that of acc-no 103 or sachin.

SQL> select acc-no, name, type from account where type = (select type from account where acc-no = 103) or type = (select type from account where name = ' Sachin');

Correlated Subquery

A correlated subquery is one whose value depends upon some variable that receives its value from outer query.

70. To find all depositors whose balance is more then the average balance in their own category.

SQL> select acc-no, balance , category from depsiter x where balance > (select avg (balance) from depositor where x.category = category);

DEPOSITOR

Acc-no	Name	balnce	catergory
105	Sachin	35k	SA
109	Saurav	25k	CA

71. To display a list of 3 highest earners in imitemp table

SQL> Select empno, ename, sal from imitemp e where 3 > (select count (*) from imitemp where sal>e.sal);

Empno	Ename	Sal
-------	-------	-----

101	Sachin	35k
102	Saurav	25k
103	Rahul	40k

	Normal Sub query	Correlated Sub query
1	A normal query is evaluated in a bottom to top manner and only once.	A correlated sub query is resolved in a top to bottom fashion
2	The innermost query is evaluated first.	The outer query is evaluated first.
3	The inner query is executed only once	The inner query is executed repeat idly, once for each candidate row.

ROWID :

Rowid is a pseudocolumn that is a part of every table. Every row in Oracle table has a unique rowid which identifies the row of data in the database.

ROW NUM :

Row num is a pseudo column. It returns the sequence no. in which a row was returned when first selected from a table. The first row has a rownum of 1, the second is 2 and so on. Rownum is used to limit the no. of rows returned by a select query.

72. To display empno and rownum from imitemp table

SQL> Select empno, rownum from imitemp;

73. Using minus operator, retrieve all employee numbers having rownum greater than 3 and less than or equal to 5.

SQL> Select empno from imitemp where rowid in (select row id from imitemp where rownum <=5 minus select row id from imitemp where rownum <3)

74. Removing duplicate records from a table using rowid and rownum

SQL>delete from imit A where rowid > (select min (rowid) from imit B where A.roll = B.roll and A.name = B.name);



**SQL>delete from imit A where rownum < (select max (rownum) from imit B where A.roll
=B.roll and A.name =B.name);**

Lab Class : 9

Working with Tables

- Tables are fundamental storage structures for data within an Oracle database.

75. To create a duplicate table as the existing table by taking some / all attributes of the existing table.

SQL>create table imit_dup as select * from imitemp;

SQL>create table imit_dup2 (ename,des,sal) as select name, des, salary from imit where dept='compsec' ;

Constraints

The constraints are used so that the Oracle does most of the work of maintaining the integrity of a database.

The constraints are

- a) Primary key
- b) Default option
- c) Foreign key
- d) Check
- e) Not Null
- f) Composite Primary Key

Primary key : When we create a primary key, Oracle automatically creates indexing on that primary key.

76. SQL > create table imit(emp_no number **primary key**, name varchar2(30), dept varchar2(10), sal number);

b) Default option : It specifies a value to be assigned to a column if a row is inserted without a value for the column.

77. SQL>create table imit (emp_no number, name varchar2(30), dept varchar2(10), sal number, doj date default sysdate);

- c) **Foreign key** : It is an attribute or group of attributes that refer to the primary key of some other table.
- d) **Check constraint** : It restricts the domain of one attribute.
- e) **Not Null** : It won't allow the null entry for one column
- f) **Composite Primary Key** : The key word unique makes more than one attributes combined to act as primary key

Example using all the above constraints :

78. SQL > create table depositor(name varchar2(30) not null, address varchar2(30) not null, acc_no number references **account** (acc_no), acc_type char(1) check(acc_type in ('s','f','c')), unique(name, address));

Enabling / Disabling constraints

79. SQL > create table account (acc_no number (5) primary key, name varchar2(30));

80. SQL > alter table account disable primary key;

81. SQL > alter table account enable primary key;

82. SQL > alter table account modify (name not null);

If the primary isn't defined for a table, we want to add it now...

83. SQL > alter table account add primary key (acc_no);

If the foreign isn't defined for a table, we want to add it now...

84. SQL > alter table depositor add constraint depkey foreign key(acc_no) references account modify (acc_no not null);

85. SQL > alter table account modify (name varchar2(20) not null);

86. SQL > alter table account drop primary key;

87. SQL > alter table depositor drop constraint depkey;

88. Roll Back - Roll back command can be used to recover deleted rows.

SQL > roll back;

Lab Class : 10

VIEWS

A view is a kind of virtual table in the database whose contents are defined by a query.

A view is a SQL*plus query that is permanently stored in the database and is assigned a name.

Creating a view :

89. SQL > create or replace view my_view as select eno, ename, salary from imitemp where salary > 10,000;

SQL> select * from my_view;

SQL> insert into my_view values(222,'Amir',2000);

SQL> select * from my_view;

SQL> select * from imitemp;

90. SQL > create or replace view myview as select eno, ename, salary from imitemp where salary > 5000 with check option;

91. SQL > insert into my_view values (111,'Aswini',3500);

Error

Advantages of View

- a) Views allow setting up different security levels for some base tables, thus restricting the access to certain data from people who don't have proper authority.
- b) Views allow the same data to be seen by different users in different ways at the same time.
- c) Views can be used to add some additional information like derived attributes.
- d) Views allow us to hide the complexities.

92. To drop a view :

SQL> drop view my_view;

Lab Class : 11

SOME MORE SQL COMMANDS

93. To make a predefined column nullable

SQL> alter table imit modify(name null);

94. To make a column nullable at the time of creation of table

SQL> create table imit(roll number, name varchar2(30) null);

95. To use default value for an attribute.

SQL> create table imit (emp_no number, name varchar2(30), dept varchar2(10), sal number, doj date default sysdate);

SQL> insert into imit values(101,'Sachin',default,20986,default);

INDEX

An index is a database structure that is created by the database server to find a row in a table quickly. Indexes are used in Oracle to improve the performance of the queries.

Once created, an index is automatically maintained by the Oracle server.

96. **SQL>** create [unique] index imitindex on imit(emp_no);

97. Displaying the indexes:

Oracle stores the list of indexes created, in a data dictionary “USER_INDEX” table.

SQL> select * from user_indexes;

98. To drop index

SQL> drop index imit_index;

index dropped

SEQUENCES

A sequence is a sequential list of numbers that is generated by the Oracle server.

99. SQL> create sequence seqimit increment by 1 start with 1000 maxvalue 1100 minvalue 1000 cache 20;

seqimit : It is a userdefined name of one sequence

start with : It specifies a number with which the sequence will begin. By default start with is maxvalue for descending sequences and minvalue for ascending sequences.

cache : It allows a preallocated set of sequence numbers to be kept in memory, for improved performance. The default is 20. the value set must be less than “maxvalue-minvalue”.

A sequence once created, is referred with nextval or curval pseudocolumn.

100. SQL> insert into imit values(seqimit.nextval, 'Saurav', 'comp', 18000, default);

SQL> select * from imit;

101. SQL> select seqimit.nextval from dual;

102. SQL> select seqimit.curval from dual;

The pseudocolumn nextval and curval can be used in the following situations :

- a) Value clauses in insert statements
- b) The select list of a select statement
- c) The set clause of an update statement

The pseudocolumn nextval and curval can't be used in the following situations:

- a) A select statement that has distinct, order by or group by clause
- b) A select statement that is used in union, minus or intersect clause

Sequence can be altered using command 'alter sequence sequence_name' followed by different parameters as given in create sequence command.

To display the attributes of a sequence, we can query the user_sequence datadictionary.

all_sequence data dictionary contains all the sequence lists owned by the user.

103. Dropping a sequence

SQL> drop sequence seqimit;

SYNONYMS

A synonym is an alternate name for any table, view or another synonym.

- Synonyms don't require any storage
- It can be created for convenience to refer to the long names or for security

104. SQL> create synonym abc for imitemp;

SQL> select * from abc;

Task for students : What is the difference between view and synonym ?

NVL function

In group functions, all the columns with a null value are ignored. The NVL function helps in substituting a value in place of null.

Syntax : NVL(column_name, value)

105. SQL> select roll,nvl(name,'NO') from imit;

Decode Function

Decode function allows the logic of **IF-THEN-ELSE** pattern.

Syntax : decode(value,if1,then1,if2,then2,...,else)

106. SQL> select roll, name, decode(stream, 'mca', 'Master of Computer', 'mba', 'Master of Business', 'Useless Courses') from imit;

Lab Class : 12

REPORT GENERATION USING SQL

Sql *Plus has commands which allows users to create reports. It uses SQL to get information from the oracle database and lets us create well formatted reports by giving us easy control over titles, column headings, column formats, reformatting of numbers and texts.

- i. The report formatting commands are all SQL *Plus commands , not SQL commands
- ii. They are not stored in SQL buffer
- iii. They remain in effect until we cancel them
- iv. SQL *plus stores many SQL commands at a time

REPORT WRITING COMMANDS

- a. Column
- b. Title
- c. Btitle
- d. Break
- e. Compute
- f. Skip
- g. Set

COLUMN

The column command can be used to specify the format of character or numeric display or column heading. The column command is given before statement.

- a. The format of character columns is specified by character 'a' followed by the width of the column
- b. The format of numeric columns is specified using 9s or 0s.
- c. The heading separator '|' is used to display the heading on next line.

107. SQL> column name heading 'Name of | student ' format a30

SQL> select * from imit;

TTITLE

The ttitle command defines a title that will appear at the top of the page.

108. SQL> ttitle center 'DATABASE OF BRILLIANT STUDENTS OF IMIT'

...

109. SQL> btitle right ' PRINCIPAL '

110. SQL> set pagesize 15

111. SQL> set linesize 20

112. SQL> set echo on/off

113. SQL> set heading on/off

114. SQL> set pause on/off

115. SQL> set wrap on/off

116. SQL> set autocommit on/off

117. SQL> set lno 24

118. SQL> set pno 1

119. SQL> set spool on/off

120. SQL> set feedback on/off

121. SQL*Plus Commands

1. Are usually entered on one line and do **not** require a ";" at the end.
2. Stay in effect until you reset them or exit the session.
3. May be entered in upper or lowercase.

SHOW

When you start a SQL*Plus session it gets it's initial settings from the LOGIN.SQL file, if one exists, in your directory path. To get a list of your current settings type:

CODE

```
SQL> show all
```

you should see a list similar to this:

```
arraysize 15
autocommit OFF
autoprint OFF
autotrace OFF
shiftinout INVISIBLE
blockterminator &quot;.&quot;; (hex 2e)
btitle OFF and is the 1st few characters of the next SELECT statement
cmdsep OFF
colsep &quot;.&quot;;
compatibility version NATIVE
concat &quot;.&quot;; (hex 2e)
copycommit 0
COPYTYPECHECK is ON
define &quot;.&quot;; (hex 26)
echo OFF
editfile &quot;afiedt.buf&quot;;
embedded OFF
escape OFF
FEEDBACK ON for 6 or more rows
flagger OFF
flush ON
heading ON
headsep &quot;|&quot;; (hex 7c)
linesize 100
lno 24
loboffset 1
long 80
longchunksize 80
newpage 1
null &quot;.&quot;;
numformat &quot;.&quot;;
numwidth 9
pagesize 24
PAUSE is OFF
pno 0
recsep WRAP
```



INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY, CUTTACK
DEPARTMENT OF MCA
MCA 1ST SEMESTER DATABASE ENGINEERING LAB

recsepchar " " (hex 20)
release 800050000
repfooter OFF and is NULL
repheader OFF and is NULL
serveroutput OFF
showmode OFF
spool OFF
sqlcase MIXED
sqlcode 0
sqlcontinue "> "
sqlnumber ON
sqlprefix "#" (hex 23)
sqlprompt "SQL> "
sqlterminator ";" (hex 3b)
suffix "sql"
tab ON
termout ON
time OFF
timing OFF
trimout ON
trimspool OFF
title OFF and is the 1st few characters of the next SELECT statement
underline "-" (hex 2d)
USER is "MY_SCHEMA_NAME"
verify ON
wrap : lines will be wrapped

STORE

If you want to save your settings after you have made changes:

CODE

```
SQL> store set my_settings_file.new create
```

will store the settings in the new file which you have named.

If you omit the file extension it will use '.sql' by default.

The keyword “create” may be changed to “replace” to overwrite an existing file or to “append” to add to an existing file.

DESCRIBE

To describe most database objects:

CODE

```
SQL> desc dual
```

Name	Null?	Type
------	-------	------

DUMMY VARCHA2(1)

START or @

To run a script:

CODE

```
SQL> start my_script.tst
```

or

CODE

```
SQL> @my_script.tst
```

If you omit the file extension, it will use '.sql' by default.

/

To run the SQL statement or PL/SQL block you typed:

CODE

```
SQL> select sysdate from dual
```

```
2
```

```
SQL> /
```

```
SYSDATE
```

```
-----
```

```
21-APR-03
```

SPOOL

To have displayed output written into an output file:

CODE

```
SQL> spool my_output.txt
```

If you omit the file extension, it will use '.lst' by default.

To stop spooling:

CODE

```
SQL> spool off
```

COLUMN

This will format a column for output:

CODE

```
SQL> column today head 'Todays Date'
SQL> select sysdate today from dual;
Todays Date
-----
21-APR-03
```

Since the heading is wider than the data you might add:

CODE

```
SQL> column today format a11
SQL> /

Todays Date
-----
21-APR-03
```

or you could have set both in one statement:

CODE

```
column today format a11 head 'Todays Date'
```

The format parameter may be used to specify a smaller width for a character string which may cause it to wrap:

CODE

```
column table_name format a15
SQL> select table_name from all_tables where owner = 'SYS' order by 1;
```

```
TABLE_NAME
-----
ACCESS$
AQ$_MESSAGE_TYP
ES
...
```

The format parameter also may assign a mask to a numeric field.

CODE

```
SQL> column num format 99,990.00
SQL> select 12345.98 num from dual;
      NUM
```

```
-----
12,345.98
```

Date masks are assigned by using the to_char function in your select statements, not in SQL*Plus. There are many optional parameters to the column command, so refer to the manual.

EDIT

To edit the contents of the SQL buffer:

CODE

```
ed
```

To edit a file:

CODE

```
ed my_script.tst
```

If you omit the file extension, it will use '.sql' by default.

GET

Used to retrieve the contents of a file into the buffer.

CODE

```
SQL> get my_script.new
1* select sysdate from dual;
```

or

CODE

```
SQL> get my_script.new nolist
SQL>
```

If you omit the file extension it will use '.sql' by default.

LIST

To list the contents of the SQL buffer:

CODE

```
SQL> l
1* select sysdate from dual;
```

PROMPT

To display a user message:

CODE

```
SQL> prompt Welcome Back!  
Welcome Back!
```

SAVE

Used to save the contents of the buffer to a file:

CODE

```
SQL> save my_sql.new create
```

This will store the buffer contents in the new file which you have named.
If you omit the file extension it will use '.sql' by default.
The keyword “create” may be changed to “replace” to overwrite an existing file or to “append” to add to an existing file.

SET

This sets the environment and there are many parameters. This is a list of some of the commonly used ones. See a manual for detailed more syntax information.

DEFINE

Sets the special character used for substitution variables (default is '&').

CODE

```
SQL> set def on ^
```

or

CODE

```
SQL> set def off
```

ECHO

Controls whether SQL*Plus commands from a command file are displayed when the command file is run. For example, if you have a file called xxx.sql which contains the SQL command:

CODE

```
select sysdate from dual;
```



Then:
CODE

```
SQL> set echo on
SQL> @xxx
SQL> select sysdate from dual;
```

```
SYSDATE
-----
21-APR-03
```

or
CODE

```
SQL> set echo off
SQL> @xxx
```

```
SYSDATE
-----
21-APR-03
```

FEEDBACK

Controls whether SQL*Plus displays the number of rows affected.

For example:

CODE

```
SQL> set feedback on
SQL> select sysdate from dual;
```

```
SYSDATE
-----
21-APR-03
```

1 row selected.

or
CODE

```
SQL> set feedback off
SQL> select sysdate from dual;
```

```
SYSDATE
-----
21-APR-03
```

HEADING

Controls whether SQL*Plus displays headings.

For example:

CODE

```
SQL> set head on
```

```
SQL> select sysdate from dual;
```

```
SYSDATE
```

```
-----
```

```
21-APR-03
```

or

CODE

```
SQL> set head off
```

```
SQL> select sysdate from dual;
```

```
21-APR-03
```

LINESIZE

Specifies the number of characters on a line.

CODE

```
SQL> set linesize 100
```

LONG

Specifies the maximum number of characters to display for a long datatype.

CODE

```
SQL> set long 200
```

NEWPAGE

Specifies the number of lines between pages. A '0' causes a formfeed.

CODE

SQL> set newpage 0

PAGESIZE

Specifies the number of lines on a page.

CODE

SQL> set pagesize 55

SCAN

Turns user variable substitution on/off.

CODE

SQL> set scan on

or

CODE

SQL> set scan off

SERVEROUTPUT

Controls whether PL/SQL blocks can print output. This is also used to set the buffer size. **Server output must be set on for DBMS_OUTPUT to work correctly.**

CODE

SQL> set serveroutput on

or

CODE

SQL> set serveroutput off

or

CODE

SQL> set serveroutput on size 100000

SHOWMODE

Determines if the before and after values of settings are displayed.

CODE

```
SQL> set show on
new: showmode BOTH
SQL> set linesize 100
old: linesize 100
new: linesize 100
SQL> set show off
old: showmode BOTH
SQL> set linesize 80
SQL>
```

TERMOUT

Determines if output from a script is displayed.
If file XXX.SQL contains:

CODE

```
select sysdate from dual;

then:
```

CODE

```
SQL> set term on
SQL> @xxx
```

```
SYSDATE
-----
22-APR-03
```

```
SQL> set term off
SQL> @xxx
SQL>
```

TIME

Controls whether time is displayed in the SQL prompt.

CODE

```
SQL> set time on
13:31:33 SQL>
```

TIMING

Controls whether the elapsed execution time displays.

CODE

```
SQL> set timing off  
SQL> @xxx
```

SYSDATE

22-APR-03

```
SQL> set timing on  
SQL> /
```

SYSDATE

22-APR-03

real: 10

TRIMOUT

Determines if trailing spaces are trimmed from lines displayed on the screen.

CODE

```
set trim on
```

or

CODE

```
set trim off
```

TRIMSPool

Determines if trailing spaces are trimmed from lines spooled to a file.

CODE



set trims on

or

CODE

set trims off

TRUNCATE

Determines if long lines are truncated.

If file XXX.SQL contains:

CODE

select '&1' hi from dual;

then:

CODE

SQL> set truncate off

SQL> set linesize 25

SQL> @xxx 123456789012345678901234567890

HI

1234567890123456789012345

67890

SQL> set truncate on

SQL> /

HI

1234567890123456789012345

VERIFY

Determines if before and after images of lines with substitution variables are displayed. If flat file XXX.SQL contains:

CODE

select '&1' hi from dual;



then:

CODE

```
SQL> @xxx Hello!  
old 1: select '&1' greeting from dual  
new 1: select 'Hello!' greeting from dual
```

```
HI  
-----  
Hello!
```

```
SQL> set verify off  
SQL> @xxx Greetings!
```

```
HI  
-----  
Greetings!
```

These settings work in combination to modify the SQL*Plus environment to suit your needs. If flat file XXX.SQL contains:

CODE

```
select '&1' msg from dual;
```

Lab Class : 13

PL/SQL

(Procedural Language/ Structured Query Language)

PL/SQL is a block structured programming language that offers application developer the ability to combine procedural logic with SQL to satisfy complex database requirements.

Advantages of PL/SQL

- Seamless SQL Access
- Integration
- Portability
- Security
- Improvement of developer's productivity
- Application performance
- Scalability
- Control Loop
- Modularity
- Error handling

PL/SQL Blocks

PL/SQL blocks can be broadly categorized into 2 types.

- i) Anonymous blocks(Block without name)
- ii) Named block(Sub program)
 - i) **Anonymous block** – These blocks are declared at the point in an application where they are to be run, e.g. **TRIGGERS** in SQL consisting of such blocks
 - ii) **Subprograms** : They are declared as procedures or functions

PL/SQL Block Structure



INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY, CUTTACK
DEPARTMENT OF MCA
MCA 1ST SEMESTER DATABASE ENGINEERING LAB

The anonymous PL/SQL block structure is made up of 3 sections, out of which 2 sections are optional.

DECLARE

Definitions of PL/SQL objects used within this block

BEGIN

Executable statements

EXCEPTION

Exception handler : i.e. what to do if executable actions cause an error

End;

Out of DECLARE, BEGIN and EXCEPTION the BEGIN section is mandatory.

New Lab Class : 14

DECLARATIVE SECTION :

Here the objects are declared. The variable, data type, cursors within the declarative section are limited to this block and can't be used outside.

```
dtype varchar2(30);
```

```
amount number (10,2) := 10000.00;
```

```
status char(1) := 'c';
```

```
balance number (10,2) default 10000.00;
```

EXECUTABLE SECTION :

It contains procedural and SQL statements that are executed when the block is run.

The statements are categorized as

- i. Assignment stmt.

```
dtype := 'HELLO'
```

- ii. Flow_of _control stmt. (GOTO, LOOP etc.)
- iii. SQL stmts.(select, insert, update & delete)
- iv. Cursor stmts.

EXCEPTION SECTION :

It contains exception handler to handle errors that occur during processing. The error handling code in the exception section is executed only when an error occurs.

END:

A PL/SQL block is terminated with end statement and a semicolon.

PL/SQL Datatypes

PL/SQL supports variety of data types for declaring variables and constants. It can be



INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY, CUTTACK
DEPARTMENT OF MCA
MCA 1ST SEMESTER DATABASE ENGINEERING LAB

- i) scalar
- ii) composite
- iii) LOB

SCALAR DATATYPES

- a) binary integer
- b) dec
- c) decimal
- d) double precision
- e) float
- f) int
- g) varchar
- h) natural
- i) number
- j) positive
- k) real
- l) char
- m) long
- n) long raw
- o) raw
- p) rowid
- q) string
- r) table
- s) varchar2
- t) date
- u) Boolean



- v) Smallint

COMPOSITE TYPES

- a) table
- b) records
- c) varraya

LOB TYPES

- a) Bfiles
- b) Blob
- c) Clob
- d) Nclob

DECLARING VARIABLES & CONSTANTS:-

Variables are declared in the declarative section of PL/SQL block.

Constants are declared by specifying the constant keyword after the datatype.

SYNTAX :

Variable_name datatype [constant][not null][:=value];

USING %TYPE:

Variables can also be declared using the column and row attributes of a table. To avoid the type and size conflict between a variable and the table column, the attribute % type is used.

A PL/SQL Program to display the records of roll 102 using %type.

Declare

Vroll student.roll%type;

Vname student.name%type;

Vmark student.mark%type;

Begin

Select roll, name, mark into vroll, vname, vmark from student where roll=102;



INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY, CUTTACK
DEPARTMENT OF MCA
MCA 1ST SEMESTER DATABASE ENGINEERING LAB

```
Dbms_output.put(vroll);  
  
Dbms_output.put(' ');  
  
Dbms_output.put_line(vname);  
  
Dbms_output.put(' ');  
  
Dbms_output.put_line(vmark);
```

End;

USING %ROWTYPE :

In case variables of entire row need to be declared, then instead of declaring them individually, the attribute %rowtype is used.

A PL/SQL Program to display the records of roll 102 using %rowtype.

Declare

```
rowvar student%rowtype;
```

Begin

```
Select roll, name, mark, city into rowvar from student where roll=102;
```

```
Dbms_output.put(rowvar.roll);  
  
Dbms_output.put(' ');  
  
Dbms_output.put(rowvar.name);  
  
Dbms_output.put(' ');  
  
Dbms_output.put(rowvar.mark);
```

End;

CONTROL STRUCTURE

1. For loop
2. while loop
3. goto
4. exit

Sample 1. FOR Loop

The following example uses a simple FOR loop to insert ten rows into a database table. The values of a loop index, counter variable, and either of two character strings are inserted. Which string is inserted depends on the value of the loop index.

Input Table

```
create table temp(col1 number, col2 number, col3 varchar2(30));
```

PL/SQL Block

```
DECLARE
    x NUMBER := 100;
BEGIN
    FOR i IN 1..10 LOOP
        IF MOD(i,2) = 0 THEN      -- i is even
            INSERT INTO temp VALUES (i, x, 'i is even');
        ELSE
            INSERT INTO temp VALUES (i, x, 'i is odd');
        END IF;
        x := x + 100;
    END LOOP;
    COMMIT;
END;
```

Output Table

```
SQL> SELECT * FROM temp;
  NUM_COL1 NUM_COL2  CHAR_COL
-----
         1        100  i is odd
         2        200  i is even
         3        300  i is odd
         4        400  i is even
         5        500  i is odd
         6        600  i is even
         7        700  i is odd
         8        800  i is even
         9        900  i is odd
        10       1000  i is even
```

Sample 2. Cursors

The following example uses a cursor to select the five highest paid employees from the emp table.

Input Table

```
SQL> SELECT ename, empno, sal FROM emp ORDER BY sal DESC;
```

ENAME	EMPNO	SAL
KING	7839	5000
SCOTT	7788	3000
FORD	7902	3000
JONES	7566	2975
BLAKE	7698	2850
CLARK	7782	2450
ALLEN	7499	1600
TURNER	7844	1500
MILLER	7934	1300
WARD	7521	1250
MARTIN	7654	1250
ADAMS	7876	1100
JAMES	7900	950
SMITH	7369	800

PL/SQL Block

```
DECLARE
    CURSOR c1 IS
        SELECT ename, empno, sal FROM emp
            ORDER BY sal DESC; -- start with highest paid
employee
    my_ename VARCHAR2(10);
    my_empno NUMBER(4);
    my_sal    NUMBER(7,2);
BEGIN
    OPEN c1;
    FOR i IN 1..5 LOOP
        FETCH c1 INTO my_ename, my_empno, my_sal;
        EXIT WHEN c1%NOTFOUND; /* in case the number requested */
                                /* is more than the total */
                                /* number of employees */
        INSERT INTO temp VALUES (my_sal, my_empno, my_ename);
        COMMIT;
    END LOOP;
    CLOSE c1;
END;
```

Output Table

```
SQL> SELECT * FROM temp ORDER BY col1 DESC;
```

NUM_COL1	NUM_COL2	CHAR_COL
5000	7839	KING
3000	7902	FORD
3000	7788	SCOTT
2975	7566	JONES
2850	7698	BLAKE

Sample 3. Scoping

The following example illustrates block structure and scope rules.

An outer block declares two variables named x and counter and loops four

times. Inside this loop is a sub-block that also declares a variable named

x. The values inserted into the temp table show that the two x's are

indeed different.

PL/SQL Block

```

DECLARE
    x NUMBER := 0;
    counter NUMBER := 0;
BEGIN
    FOR i IN 1..4 LOOP
        x := x + 1000;
        counter := counter + 1;
        INSERT INTO temp VALUES (x, counter, 'in OUTER loop');
        /* start an inner block */
        DECLARE
            x NUMBER := 0; -- this is a local version of x
        BEGIN
            FOR i IN 1..4 LOOP
                x := x + 1; -- this increments the local x
                counter := counter + 1;
                INSERT INTO temp VALUES (x, counter, 'inner loop');
            END LOOP;
        END;
    END LOOP;
END LOOP;

```

```
COMMIT;  
END;
```

Output Table

```
SQL> SELECT * FROM temp ORDER BY col2;
```

NUM_COL1	NUM_COL2	CHAR_COL
1000	1	in OUTER loop
1	2	inner loop
2	3	inner loop
3	4	inner loop
4	5	inner loop
2000	6	in OUTER loop
1	7	inner loop
2	8	inner loop
3	9	inner loop
4	10	inner loop
3000	11	in OUTER loop
1	12	inner loop
2	13	inner loop
3	14	inner loop
4	15	inner loop
4000	16	in OUTER loop
1	17	inner loop
2	18	inner loop
3	19	inner loop
4	20	inner loop

Lab Class : 15

PL/SQL Programs

1. Display the data of a table using %type.

Declare

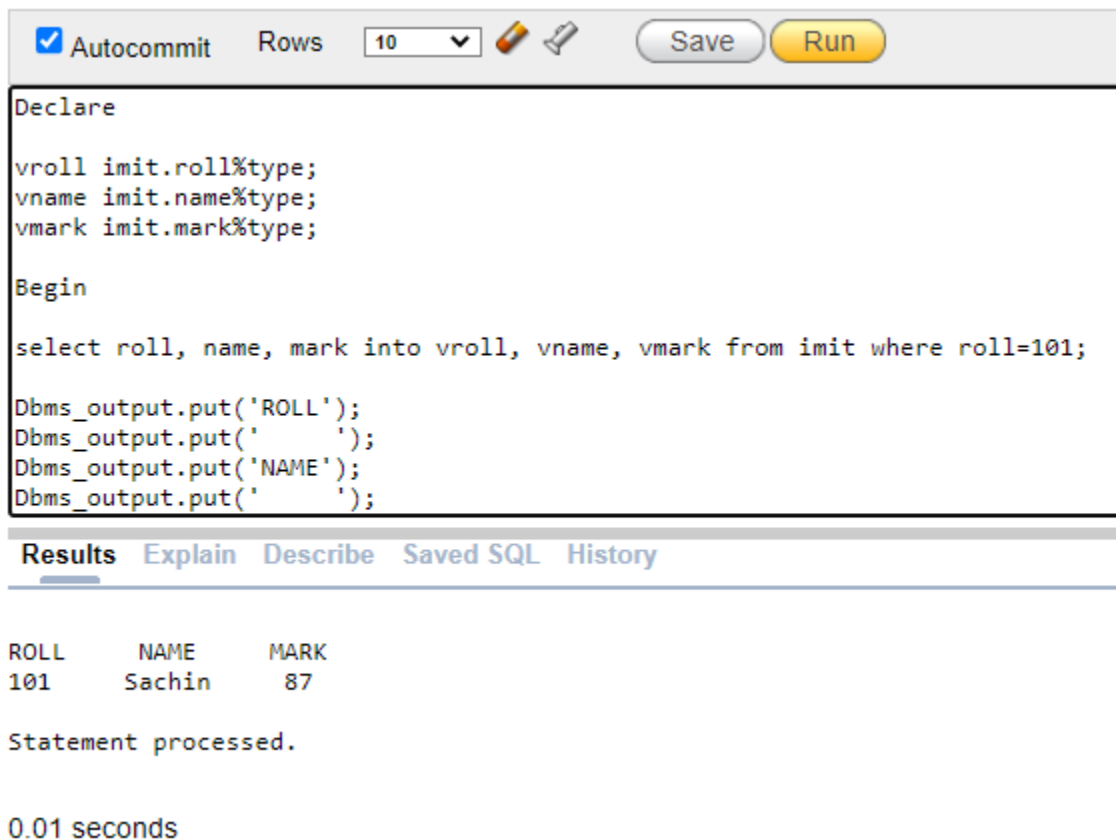
```
vroll imit.roll%type;  
vname imit.name%type;  
vmark imit.mark%type;
```

Begin

```
select roll, name, mark into vroll, vname, vmark from imit where roll=101;
```

```
Dbms_output.put('ROLL');  
Dbms_output.put(' ');  
Dbms_output.put('NAME');  
Dbms_output.put(' ');  
Dbms_output.put_line('MARK');
```

```
Dbms_output.put(vroll);  
Dbms_output.put(' ');  
Dbms_output.put(vname);  
Dbms_output.put(' ');  
Dbms_output.put_line(vmark);  
End;
```



The screenshot shows a SQL IDE window with a toolbar at the top containing a checked 'Autocommit' checkbox, a 'Rows' dropdown set to '10', and 'Save' and 'Run' buttons. The main text area contains the following PL/SQL code:

```
Declare
vroll imit.roll%type;
vname imit.name%type;
vmark imit.mark%type;

Begin
select roll, name, mark into vroll, vname, vmark from imit where roll=101;

Dbms_output.put('ROLL');
Dbms_output.put(' ');
Dbms_output.put('NAME');
Dbms_output.put(' ');
```

Below the code area, there is a tabbed interface with 'Results' selected. It displays the following output:

ROLL	NAME	MARK
101	Sachin	87

Below the table, the text 'Statement processed.' and '0.01 seconds' are displayed.

2. Display the data of a table using %rowtype.

```
Declare
vrow imit%rowtype;

Begin
select roll, name, mark into vrow from imit where roll=102;

Dbms_output.put('ROLL');
Dbms_output.put(' ');
Dbms_output.put('NAME');
Dbms_output.put(' ');
Dbms_output.put_line('MARK');

Dbms_output.put(vrow.roll);
Dbms_output.put(' ');
Dbms_output.put(vrow.name);
Dbms_output.put(' ');
Dbms_output.put_line(vrow.mark);
End;
```

☒ Autocommit Rows  

```

Declare
vrow imit%rowtype;

Begin
select roll, name, mark into vrow from imit where roll=102;

Dbms_output.put('ROLL');
Dbms_output.put(' ');
Dbms_output.put('NAME');
Dbms_output.put(' ');
Dbms_output.put_line('MARK');

Dbms_output.put(vrow.roll);
Dbms_output.put(' ');

```

Results Explain Describe Saved SQL History

ROLL	NAME	MARK
102	Saurav	97

Statement processed.

0.01 seconds

3. Insert records into a table using PL/SQL.

Step –1 : create table temp(col1 number, col2 number, col3 varchar2(30));

Step – 2:

DECLARE

x NUMBER := 100;

BEGIN

FOR i IN 1..10 LOOP

IF MOD(i,2) = 0 THEN -- i is even

INSERT INTO temp VALUES (i, x, 'i is even');

ELSE

INSERT INTO temp VALUES (i, x, 'i is odd');

END IF;

x := x + 100;

END LOOP;

COMMIT;

END;

Step – 3:

select * from temp;

```
create table temp(col1 number, col2 number, col3 varchar2(30));
DECLARE
  x NUMBER := 100;
BEGIN
  FOR i IN 1..10 LOOP
    IF MOD(i,2) = 0 THEN      -- i is even
      INSERT INTO temp VALUES (i, x, 'i is even');
    ELSE
      INSERT INTO temp VALUES (i, x, 'i is odd');
    END IF;
    x := x + 100;
  END LOOP;
  COMMIT;
END;
```

Results Explain Describe Saved SQL History

COL1	COL2	COL3
1	100	i is odd
2	200	i is even
3	300	i is odd
4	400	i is even
5	500	i is odd
6	600	i is even
7	700	i is odd
8	800	i is even
9	900	i is odd
10	1000	i is even

Cursor

4. PL/SQL program using cursor to insert into table temp2 records having highest 5 marks from imit table.

Step – 1:

select * from imit

Results Explain Describe Save

ROLL	NAME	MARK
101	Sachin	87
102	Saurav	97
103	Ajay	77
104	Shahrukh	57
105	Nihar	71
106	Rahul	84
107	Pupun	75
108	Ram	92
109	Sai	58
110	Sajan	95

10 rows returned in 0.00 seconds

Step – 2:

create table temp2(roll number, name varchar2(30), mark number)

Step – 3:

DECLARE

CURSOR c1 is

SELECT roll, name, mark FROM imit ORDER BY mark DESC;

my_roll NUMBER;

my_name VARCHAR2(30);

my_mark NUMBER;

BEGIN

OPEN c1;

FOR i IN 1..5 LOOP

FETCH c1 INTO my_roll, my_name, my_mark;

EXIT WHEN c1%NOTFOUND;

INSERT INTO temp2 VALUES (my_roll, my_name, my_mark);

COMMIT;

END LOOP;

CLOSE c1;

END;

Step – 3:

select * from temp2

☒ Autocommit Rows 10 Save Run

```

DECLARE
  CURSOR c1 is
  SELECT roll, name, mark FROM imit ORDER BY mark DESC;
  my_roll NUMBER;
  my_name VARCHAR2(30);
  my_mark NUMBER;
BEGIN
  OPEN c1;
  FOR i IN 1..5 LOOP
    FETCH c1 INTO my_roll, my_name, my_mark;
    EXIT WHEN c1%NOTFOUND;
    INSERT INTO temp2 VALUES (my_roll, my_name, my_mark);
    COMMIT;
  END LOOP;
  
```

Results Explain Describe Saved SQL History

ROLL	NAME	MARK
102	Saurav	97
110	Sajan	95
108	Ram	92
101	Sachin	87
106	Rahul	84

5 rows returned in 0.00 seconds [Download](#)